CCS3002: NUMERICAL METHODS

The Bisection Method

How We Find Roots Step-by-Step (Theory & Python App)

In this presentation, we will cover:

1. Why we use numerical methods and how errors happen in computers
2. How the Bisection Method works visually with sign changes (+ve / -ve)
3. Solving classic exam problems with our interactive Python Solver App

Introduction: Why Do We Need Numerical Methods?

From analytical algebra to computer approximation

The Problem with Normal Algebra

- In high school, we solve equations with exact formulas:
  $2x + 4 = 0 \quad => \quad x = -2$
  $x^2 - 5x + 6 = 0 \quad => \quad$ quadratic formula

- But in real engineering and science, equations look like:
  $x^3 - x - 11 = 0$
  $x - \cos(x) = 0$
  $x \cdot e^x = 2$

- There is NO simple formula to isolate 'x'!
  We call this a lack of an analytical solution.

The Numerical Solution

- Instead of solving in one magic step, computers take repeated smart guesses (iterations).

- Each guess gets closer and closer to the true root until the error is small enough to accept.

- Why this matters today:
  ✔ Computers do thousands of guesses per second.
  ✔ It works on almost ANY continuous equation!
  ✔ Gives us practical answers for engineering problems.

Key Takeaway: Numerical methods give us practical, highly accurate solutions when exact math is impossible.

Where Do Errors Come From?

Why calculations are approximations and why tiny errors matter

1. Model Errors

Reality is too complex, so equations simplify things.

- Example: Ignoring air resistance when calculating free fall.
- We make assumptions to make the math solvable.

2. Round-off Errors

Computers store numbers in finite bits (32 or 64 bits).

- A computer cannot store 1/3 = 0.33333333... forever.
- Numbers like π and √2 get cut off, creating tiny round-off errors.

3. Truncation Errors

Cutting off an infinite mathematical series after a few terms.

- Example: Taylor series approximation of e^x.
- Using 3 or 4 terms leaves a small residual error.

Famous Real-World Lesson: The 1991 Patriot Missile Failure

A radar system tracked time by adding 0.1 seconds repeatedly. Because 0.1 in binary cannot be stored exactly (round-off error), the computer accumulated a tiny error of 0.000000095 seconds every tick. After 100 hours of continuous operation:

Accumulated Error = 0.34 seconds => Radar tracked target off by over 500 meters!

Result: Failed to intercept an incoming Scud missile. Lesson: In computing, understanding error is a matter of life and death.

The 4 Types of Numerical Errors

Exact vs. Approximate, Absolute vs. Relative (Lecture Slide 8)

Think of it this way:

- Exact Error: Compares your guess to the TRUE answer (only possible when someone tells you the true answer in advance!).
- Approximate Error: Compares your NEW guess to your OLD guess (always available during computer iterations!).
- Relative Error: Turns the error into a percentage (%) so you know how accurate it is regardless of scale.

Summary Table (Standard Lecture Formula Reference)

Error Type	Formula	When Available	Typical Use in App
Exact Error (Et)	| X_true - X_approx |	Rare (needs true value)	Theoretical benchmarking
Approximate Error (Ea)	| X_new - X_old |	Always during iterations	Measures step movement
Relative Exact Error (et)	(| X_true - X_approx | / | X_true |) × 100%	Rare (needs true value)	Reports final accuracy %
Relative Approx. Error (ea)	**(| X_new - X_old | / | X_new |) × 100%**	**Always during iterations**	**OUR STOPPING CRITERION!**

What is a Root? (Bolzano's Sign Change Rule)

Finding where the curve crosses zero (Lecture Slide 10)

What is a Root?

A root of f(x) is simply the point where:

$$f(x) = 0$$

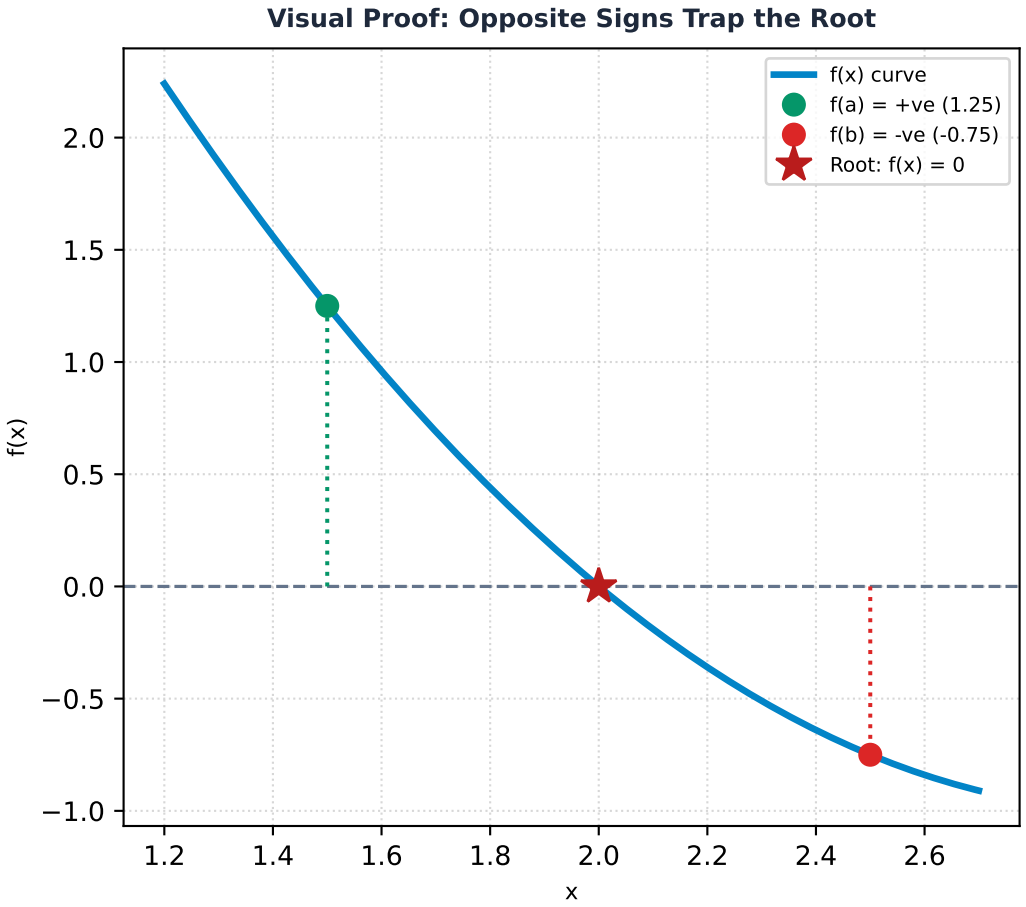
On a graph, it is where the curve crosses the horizontal axis.

Bolzano's Intermediate Value Theorem:

If a curve starts below the axis (negative) and ends above the axis (positive), it CANNOT teleport across without crossing zero!

Mathematically:
If $f(a) < 0$ and $f(b) > 0$ (opposite signs),
Then there MUST be at least one root between a and b!

This gives us a 100% guarantee that a solution is trapped inside [a, b].



How the Bisection Method Works

The simple 3-step loop: Cut, Check Sign, Discard Half

Step 1: Find Midpoint

Cut the interval in half!

x_mid = (a + b) / 2

This midpoint is your current best approximation of the root.

Step 2: Check Sign of f(x_mid)

Plug the midpoint into the equation:

- Is f(x_mid) == 0? You hit the exact root! Stop.
- Is f(x_mid) positive (+ve) or negative (-ve)?

Step 3: Keep the Bracket

Pick the half that has OPPOSITE signs!

- If sign of f(x_mid) == sign of f(a):
  The root is in the right half -> Replace a = x_mid.
- Else replace b = x_mid. Throw away the rest!

When Do We Stop? (Stopping Criteria — Lecture Slide 9)

In real exams and computer software, we stop iterating when:
1. Relative Error falls below tolerance: ea = (|x_new - x_old| / |x_new|) × 100% ≤ Tolerance
2. Number of decimal places is reached: e.g., accurate to 3 decimal places (error < 0.0005)
3. Maximum iterations reached: protects against infinite loops.

Worked Example: $x^2 - 6x + 8 = 0$

Step-by-step walkthrough directly from Lecture Slide 11 & 12

Problem Statement (Lecture Example 1):

Solve $f(x) = x^2 - 6x + 8 = 0$ in $[0, 3]$

Initial Bracket Check:
- $f(0) = 0^2 - 6(0) + 8 = +8$ (+ve)
- $f(3) = 3^2 - 6(3) + 8 = -1$ (-ve)

Opposite signs! Root exists in [0, 3].

Iteration 1:
- Midpoint: $x_1 = (0 + 3)/2 = 1.5$
- $f(1.5) = 1.5^2 - 6(1.5) + 8 = +1.25$ (+ve)
- Since f(1.5) is +ve, replace a = 1.5. New interval: [1.5, 3].

Iteration 2:
- Midpoint: $x_2 = (1.5 + 3)/2 = 2.25$
- Error = $|2.25 - 1.5| / 2.25 \times 100\% = 33.33\%$

Lecture Iteration Table (Lecture Slide 12)

a (+ve)	b (-ve)	x_n = (a+b)/2	Error %	f(x_n)
0	3	1.5	---	+ve
1.5	3	2.25	33%	-ve
1.5	2.25	1.875	20%	+ve
1.875	2.25	2.0625	9.091%	-ve
1.875	2.0625	1.96875	4.76%	+ve
1.96875	2.0625	2.015625	2.32%	-ve

Our Python Solver App (bisection.py)

How the app makes solving effortless, safe, and visual

1. Smart Expression Parser

Type equations naturally without strict Python syntax!

- Implicit Multiplication:
  Type '2x' or '2sin(x)' — the app auto-converts to '2*x'.
- Powers with Caret '^':
  Type 'x^3 - 7x + 1' — converted safely to 'x**3'.
- Safe AST Sandbox:
  Blocks dangerous inputs; accepts all standard functions (sin, cos, exp, log, sqrt, pi).

2. Complete Terminal Table

Prints every iteration clearly with ASCII borders:

- 6 Informative Columns:
  Iter | Lower (a) | Higher (b) | Midpoint (c) | Approx Err (Ea) | Rel Approx (ea) %
- No Hidden Rows:
  Prints all iterations from 1 to N.
- Clear Stop Marker:
  Flags the converged row with '<-- [STOP]'.

3. 2-Panel Graphical Plot

Visual verification saved directly to bisection_plot.png:

- Left Panel: Root Localization
  Shows the function curve, shaded bracket [a, b], and marks the root with a star.
- Right Panel: Error Decay Trajectory
  Plots log error decay vs. iteration against the tolerance line.
- Interactive GUI window opens automatically!

Solving Exam Exercise 3: x³ - 7x + 1 = 0

Finding the smaller positive root to 3 decimal places (Exercise Slide 16)

Running it in the App:

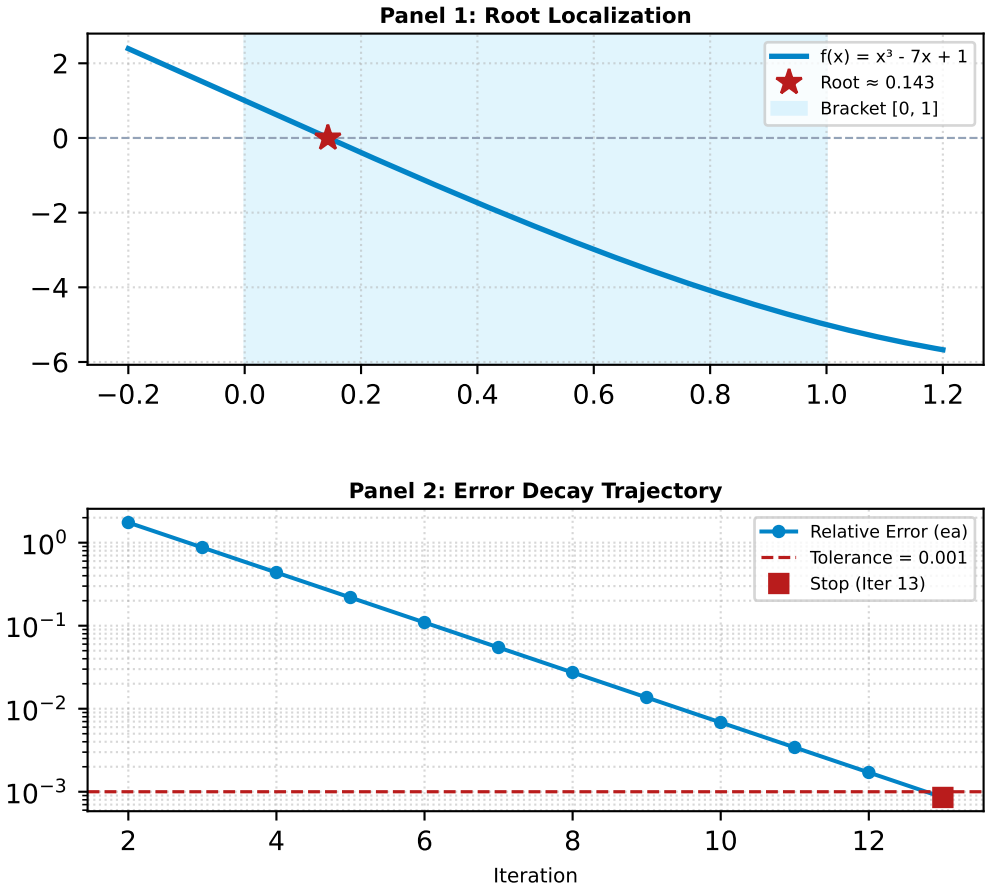
Input: f(x) = x^3 - 7x + 1, Interval: [0, 1], Tolerance: 0.001

```
+================================================================================================+
|                              3. CONVERGENCE SUMMARY & ERROR ANALYSIS                           |
+================================================================================================+
  Problem Specification:
    * Function f(x)       : x^3 - 7x + 1
    * Initial Range [a, b]: [0, 1]
    * Error Tolerance     : 0.001 (0.1%)
    * Stopping Criterion  : Relative Approximate Error (ea <= tolerance)

  Solution Results:
    * Approximate Root (c): 0.14318848
    * Function Value f(c) : +6.164488e-04  (effectively zero)
    * Iterations Required : 13
    * Termination Reason  : Reached relative tolerance or exact root

  Error Analysis (4 Numerical Errors):
    1. Relative Approx Error (ea) [Stopping Metric]: 8.5251e-04 (8.5251e-02%)
    2. Absolute Approx Error (Ea)                  : 1.220703e-04
    3. Absolute Exact Error  (Et = |x_true - c|)   : 8.884344e-05
    4. Relative Exact Error  (et = |x_true - c|/|x_t|): 6.2008e-04 (6.2008e-02%)
+================================================================================================+
```

Auto-Generated Plot (bisection_plot.png)



Summary & Exam Cheat Sheet

Everything you need to know on one slide

Golden Rules of Bisection

1. Always check initial signs first: f(a)·f(b) < 0.
2. Midpoint is always: c = (a + b) / 2.
3. Replace the endpoint that has the SAME sign as f(c).
4. Calculate error using consecutive midpoints:
   ea = |c_new - c_old| / |c_new| × 100%.
5. '3 decimal places' means tolerance = 0.0005 (|c_new - c_old| < 0.0005).

Why Bisection is Loved

- 100% Reliable: It NEVER fails or diverges if bracket is valid.
- No Calculus: Doesn't need f'(x) (unlike Newton-Raphson).
- Fast Shrinking: Halves interval width every step (shrinks by >1,000x in 10 steps!).
- Perfect for starting other faster numerical methods.

Using the Python App

1. Open terminal: python bisection.py
2. Enter function: e.g. x^3 - 7x + 1 or 2x + sin(x)
3. Enter interval: e.g. 0, 1
4. Enter tolerance: e.g. 0.001 (or hit Enter for default)
5. View complete 102-char iteration table & 2-panel plot window!